

0.54.2. Trois lettres, trois carrés — doublé, deux positions

Les trois carrés du 0.54, puis celui du D : quatre lettres.

1. Ce qu'il faut comprendre

C'est le 0.54 et le 0.54.1 réunis : les trois carrés, puis le quatrième.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2175 octets** de cartouche.

1. UNE VARIABLE — RONDE

```
Carre ronde = { 15, 8, ALPHABET[2], 2, 1, 250, 2 }; // 1e C
```

2. UNE VARIABLE — BAS

```
Carre bas = { 15, 14, ALPHABET[3], 2, 1, 250, 1 }; // 1e D
```

3. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    carre(4, 4, ALPHABET[0], 1, 1, 250, 1); // 1e A
    carre(9, 9, ALPHABET[1], 3, -1, 100, 1); // 1e B
    carre(ronde); // 1e C
    carre(bas); // 1e D

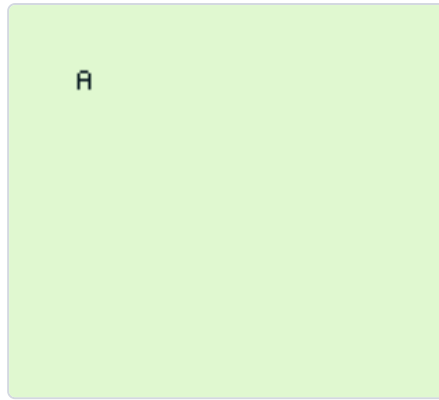
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

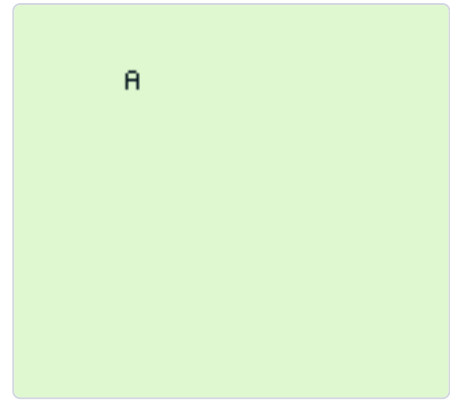
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



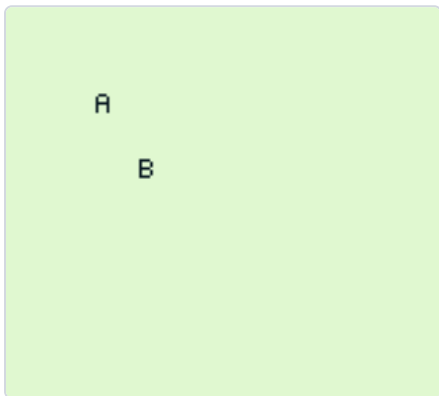
la toute première image — la console vient de démarrer — image 1



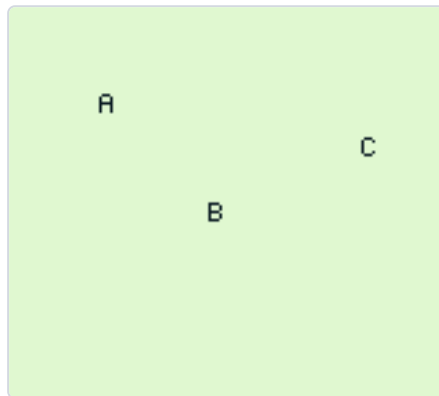
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A, le B, le C, puis le D font leur carré.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A finit en (4, 4) (4,4)

✓ le B finit en (9, 9) (9,9)

✓ le D finit en (15, 14) (15,14)

Ensuite : **0.55. La vitesse des déplacements : vitesse** — niveau 0 sur 10. · [gameboy3](#) — écrire une cartouche Game Boy en C++